

MODULE 14 · DOMAIN 4 - OPERATIONAL EFFICIENCY AND OPTIMIZATION

Cost and Performance

Tasks 4.1 and 4.2. Token economics, cost attribution, caching, tiering and latency.

DOMAIN 4 of 5 EXAM WEIGHT 12% MODULE 14 of 22

Everything in this domain reduces to one unit. **Tokens**. Cost is tokens, throughput quotas are tokens, context limits are tokens, and latency is largely tokens generated.

Where the token bill actually comes from

- ◆ **Output tokens cost several times more than input tokens.** Limiting response length is disproportionately effective.
- ◆ **Conversations grow quadratically.** Every turn resends every previous turn, as module 3 established.
- ◆ **Retrieved context is usually the largest input component,** and top-k is the dial that controls it.
- ◆ **maxTokens caps output only.** It does nothing about a 40,000-token prompt.
- ◆ **Estimate before you build.** Tokens per request times requests per day times price is arithmetic you can do on a napkin.

ON THE EXAM

Skill 4.1.1 names token estimation and tracking first. Most cost surprises are not exotic; they are a team that never multiplied out the expected volume.

Six ways to send and generate fewer tokens

- ◆ **Context window optimisation.** Send what the answer needs, not all you retrieved.
- ◆ **Context pruning.** Drop chunks below a relevance threshold instead of all of top-k.
- ◆ **Prompt compression.** Trim boilerplate that repeats on every call.
- ◆ **Response size controls.** Ask for three bullets, pay for three bullets.
- ◆ **Summarise conversation history** rather than resending it verbatim.

ON THE EXAM

Context pruning is the highest-value one in a RAG system. Retrieving ten chunks and sending only the four above threshold cuts input tokens sharply with little accuracy loss.

A support assistant retrieves top-20 chunks from a knowledge base and sends all of them with every question. Answers are good. The monthly Bedrock bill is dominated by input tokens, and analysis shows the retrieved context averages 14,000 tokens per request against a 300-token question.

Relevance scores show that beyond roughly the sixth chunk, scores drop sharply. The team must reduce cost substantially without a measurable drop in answer quality, and has proposed switching to a cheaper model.

What should the team do first?

- ☐ A Switch to a cheaper model with a lower per-token price
- ☐ B Retrieve top-20 for recall, then prune to the chunks above a relevance threshold before building the prompt, and validate against a golden set
- ☐ C Reduce maxTokens so responses are shorter
- ☐ D Reduce retrieval to top-5 chunks

◆ PAUSE AND DECIDE

What should the team do first?

A Cheaper model

Reduces the unit price of sending 14,000 largely irrelevant tokens. It also risks the quality constraint the scenario states.

C Lower maxTokens

Caps output when the bill is dominated by input. It attacks the smaller side of the ledger and shortens answers.

B Retrieve wide, prune by relevance, validate

Keeps recall high while cutting input tokens by roughly two thirds. The relevance-score evidence is handed to you in the scenario.

D Retrieve top-5

Close, and it discards recall as well as tokens. Retrieving wide and pruning keeps the occasional good chunk that ranked seventh.

THE SIGNAL

"Beyond roughly the sixth chunk, scores drop sharply" is the measurement being handed to you. B and D reach a similar token count; B keeps the recall that a fixed top-5 throws away.

You cannot optimise what you cannot attribute. In most organisations, the Bedrock line on the bill is **one number for everybody.**

Two kinds of inference profile

SYSTEM-DEFINED

Cross-Region routing

- Prefixes us., eu., apac., global.
- Created and maintained by AWS
- Solves capacity and residency
- Module 1 covered this one

APPLICATION INFERENCE PROFILE

Cost attribution

- You create it, and you tag it
- Wraps a model or a system-defined profile
- Solves "which team spent this?"
- Tags become cost allocation tags

THE TIEBREAKER

Same phrase, different jobs. If a scenario is about capacity or residency it means the first. If it is about attributing spend to a team, tenant or workload it means the second.

Application inference profiles, precisely

- ◆ Create a profile per team, tenant or workload, **tag it**, and have that application invoke the profile ARN.
- ◆ Billed dollars flow to **Cost Explorer and the Cost and Usage Report**, split by your tags.
- ◆ **The finest grain is per usage type per day.** This is not per-request cost.
- ◆ For per-prompt token detail, use **per-request metadata tagging with model invocation logs** instead.
- ◆ Tags must be **activated** as cost allocation tags, take up to 24 hours to appear, and are **not retroactive**.

ON THE EXAM

Two traps live here. It is daily aggregate rather than per request, and activation is not backdated, so the month before you switched it on stays unattributed forever.

A SaaS platform serves 400 business customers from a shared Bedrock deployment. Finance cannot determine per-customer AI cost and suspects that a small number of customers are unprofitable. All applications currently invoke model IDs directly.

Finance needs cost broken down per customer in Cost Explorer for monthly billing review. A separate product team also wants to see token consumption for individual requests to identify expensive query patterns.

Which combination meets both needs?

- A Create tagged application inference profiles per customer for Cost Explorer attribution, and enable model invocation logging with per-request metadata for token-level detail
- B Tag the Lambda functions with a customer identifier and use cost allocation tags on Lambda
- C Create tagged application inference profiles per customer, which provides both the monthly breakdown and per-request token detail
- D Build a Lambda wrapper that counts tokens per call and writes them to a DynamoDB table with a customer key

Which combination meets both needs?

- A

 Tagged profiles plus invocation logging metadata
Profiles give Cost Explorer the monthly per-customer dollars; per-request metadata in invocation logs gives the token-level detail profiles cannot.
- B

 Tag the Lambda functions
Attributes Lambda cost, not Bedrock cost. The model spend is the line finance cares about and it stays a single number.
- C

 Profiles alone
The near miss. Profiles aggregate to per usage type per day, so they answer finance and cannot answer the product team's per-request question.
- D

 Lambda wrapper counting tokens
Builds a metering service AWS already provides, and the numbers will not reconcile with the actual bill.

THE SIGNAL

Two requirements at two granularities. Option C is right for one of them, which is exactly why knowing the "per usage type per day" limit matters.

The cheapest invocation is the one you never make. Caching is the only lever on this list that reduces cost and latency **at the same time.**

What each caching layer actually does

LAYER	MATCHES ON	USE IT WHEN
Prompt caching	A stable prefix, inside Bedrock	Long shared context reused within minutes
Exact-match cache	A hash of the whole request	Identical requests repeat, such as API clients
Semantic cache	Embedding similarity to past queries	Humans ask the same thing in different words
Edge cache	URL or key at CloudFront	Public, identical responses served widely

ON THE EXAM

The exam guide names all four: prompt caching, deterministic request hashing, semantic caching and edge caching. Result fingerprinting is the exact-match idea under another name.

Exact-match or semantic caching?

EXACT MATCH

Hash the request

- Deterministic, trivially correct
- DynamoDB, ElastiCache or MemoryDB
- Almost no hit rate on human phrasing
- Cheap to build and to run

SEMANTIC

Embed the query

- Matches paraphrase and rewording
- A vector store holds question embeddings
- You set a similarity threshold
- Too loose and you serve the wrong answer

THE TIEBREAKER

"Users ask similar questions in different ways" means semantic. A key-value cache will sit at a near-zero hit rate, because no two people phrase a question identically.

A public sector helpline assistant answers benefit eligibility questions. Analysis shows that roughly 70% of questions are variations of about 40 underlying topics, phrased differently by each caller. Responses do not vary by caller and change only when policy is updated.

Cost and latency both need to fall. An engineer has implemented an ElastiCache lookup keyed on a hash of the question text and reports a 2% hit rate.

What should the team implement?

- ☐ A Increase the ElastiCache TTL so entries survive longer
- ☐ B A semantic cache: embed each question, look for a stored question above a similarity threshold, and return the cached answer on a hit
- ☐ C Enable Bedrock prompt caching on the system prompt
- ☐ D Pre-compute answers to the 40 topics and serve them from CloudFront

◆ PAUSE AND DECIDE

What should the team implement?

- A

 Longer TTL
The 2% hit rate is not an expiry problem. Entries are not being matched because no two callers type the same string.
- C

 Prompt caching on the system prompt
Saves the shared prefix and still invokes the model on every question. It reduces the bill somewhat and avoids no invocations.
- B

 Semantic cache on question embeddings
70% of traffic collapses onto 40 topics once you match on meaning rather than characters. Answers are caller-independent, so a cached answer is safe to reuse.
- D

 Pre-compute 40 answers at the edge
Genuinely appealing, and it requires mapping an arbitrary question to one of 40 topics. That mapping is a semantic match, so you have built option B with extra steps.

THE SIGNAL

"Phrased differently by each caller" plus a 2% hit rate on a hash is the semantic-cache signal stated twice. And "responses do not vary by caller" is what makes caching safe here at all.

The complete cost lever inventory

LEVER	TYPICAL EFFECT	YOU GIVE UP
Context pruning	Cuts input tokens sharply	Some recall, if tuned badly
Prompt caching	Up to 90% cost, 85% latency	Nothing, if the prefix is stable
Semantic caching	Avoids the invocation entirely	Freshness, and threshold risk
Model tiering or cascading	Most traffic on a cheap model	A check to build and maintain
Intelligent Prompt Routing	Up to 30% cost	Two models from one family only
Batch inference	50% discount	Any completion time guarantee
Flex tier	50% discount	Latency under load

ON THE EXAM

Read the third column. Every lever except prompt caching costs you something, and the scenario usually states which of those costs is unacceptable.

When the answer is to change nothing

- ◆ **Bursty traffic plus "optimise cost" usually means on-demand is already correct.**
- ◆ **Batch inference has no completion-time guarantee.** A downstream deadline makes 50% costly.
- ◆ **Provisioned Throughput wins only above a break-even.** Below it, on-demand is cheaper.
- ◆ Calculate the break-even. Committed capacity is a bet on sustained utilisation.
- ◆ The commonest real waste is **invocations that need not have happened.**

ON THE EXAM

Module 1 made this from the throughput side. From the cost side: 'bursty' or 'unpredictable' in a scenario argues against every form of reservation.

A retailer's Bedrock spend has grown 40% in a quarter. Traffic is highly variable: quiet overnight, sharp peaks at lunchtime and in the evening, and unpredictable spikes during promotions. The workload is a customer-facing chat assistant with sub-two-second latency expectations.

An architect has proposed purchasing Provisioned Throughput sized to the peak to reduce the per-token rate. Investigation shows the assistant resends full conversation history on every turn, and sessions average 22 turns.

What is the most effective response?

- ☐ A Purchase Provisioned Throughput sized to the observed peak
- ☐ B Address the quadratic conversation growth with history summarisation and a rolling window, and keep on-demand pricing
- ☐ C Move the workload to batch inference for the 50% discount
- ☐ D Set serviceTier to flex to obtain the 50% discount

◆ PAUSE AND DECIDE

What is the most effective response?

- A

 Provisioned Throughput at peak
Pays for peak capacity around the clock against traffic that is quiet overnight. Reservation is the wrong instrument for unpredictable load.
- B

 Fix the conversation growth, stay on-demand
22 turns resending everything is the actual cause of the increase. It cuts tokens across every request without touching latency or pricing mode.
- C

 Batch inference
Asynchronous with no completion guarantee, applied to a customer waiting for a chat reply. It cannot serve this workload at all.
- D

 serviceTier=flex
Trades latency under load for 50%. The workload has a sub-two-second expectation and peaks are exactly when Flex slows down.

THE SIGNAL

The scenario diagnoses itself in its last sentence. Every pricing-mode option is wrong for a bursty, latency-sensitive workload, which leaves the one option that reduces tokens.

Latency is a different currency, and it has **three separate measurements**. Confusing them is why teams optimise the wrong thing for weeks.

Which number is the complaint about?

METRIC	THE COMPLAINT SOUNDS LIKE	WHAT MOVES IT
Time to first token	"It feels slow", "users abandon"	Streaming, prompt caching, shorter prompts
Total latency	"The answer takes too long"	Smaller model, fewer output tokens, latency-optimised inference
Throughput	"It falls over at peak", throttling	Concurrency, capacity, cross-Region inference, queueing

ON THE EXAM

TTFT is the primary user-experience metric for anything streamed. Total latency is what matters when a machine consumes the output and nobody is watching it arrive.

The latency levers the guide names

- ◆ **Pre-computation for predictable queries.** If you know it will be asked, answer it before it is asked.
- ◆ **Latency-optimised Bedrock inference** for time-sensitive applications, on the models that support it.
- ◆ **Parallel requests for complex workflows.** Independent sub-tasks should not run in sequence.
- ◆ **Streaming** so time to first token stops being the same as total latency.
- ◆ **Vector query optimisation:** retrieval sits on the critical path of every RAG request, and it is measurable separately.

ON THE EXAM

Retrieval latency is the component teams forget to measure. A RAG request that takes three seconds may be spending one of them in the vector store before the model starts.

Fanning out independent model calls

01

Split

Decompose the task into genuinely independent sub-tasks.

02

Map

A Step Functions Map state runs them concurrently.

03

Bound

Set MaxConcurrency so you do not throttle yourself.

04

Join

Aggregate results, then a final call composes the answer.

ON THE EXAM

Total latency becomes the slowest branch rather than the sum of all branches. The constraint is your tokens-per-minute quota, which is why MaxConcurrency matters.

A due diligence tool analyses a company across six independent dimensions: financial, legal, technical, market, operational and regulatory. Each dimension retrieves its own documents and makes one model call. A final call composes a summary from the six.

The tool currently runs the six sequentially and takes 47 seconds end to end. Analysts describe it as unusable. Each individual dimension takes six to nine seconds. Analysts do not need the intermediate results; they read the final summary.

What is the most effective change?

- ☐ A Stream the final summary so results appear as they are generated
- ☐ B Run the six dimensions concurrently with a Step Functions Map state with bounded concurrency, then compose the summary
- ☐ C Switch to a smaller, faster model for all seven calls
- ☐ D Combine all six dimensions into a single large prompt with one model call

◆ PAUSE AND DECIDE

What is the most effective change?

- A

 Stream the final summary
Improves time to first token on the last call only. The analyst still waits roughly 40 seconds before that call even begins.
- B

 Step Functions Map with bounded concurrency
The dimensions are independent, so total time becomes the slowest branch plus the summary. Roughly 47 seconds becomes roughly 15.
- C

 Smaller model everywhere
Reduces each of seven sequential calls and keeps the sequence. It trades analysis quality for a saving that parallelism gives for free.
- D

 One combined prompt
One enormous prompt across six document sets, with quality dropping as the model divides attention. It also removes the parallelism opportunity.

THE SIGNAL

"Six independent dimensions" run "sequentially" is the whole question. Independent work in sequence is latency you are choosing to pay.

Throughput, capacity and the last few per cent

- ◆ **Batching** raises throughput on self-hosted endpoints by keeping the GPU busy across concurrent requests.
- ◆ **Capacity planning** means sizing against summed true peaks, as module 8's scenario 2 showed.
- ◆ **Auto-scaling** is only useful if a new instance becomes useful before the spike ends. Module 9's point.
- ◆ **Parameters affect latency too.** A high maxTokens invites a long answer you then wait for.
- ◆ **A/B test the change.** Skill 4.2.4 names it, and it is the only way to know an optimisation cost you nothing.

ON THE EXAM

Every optimisation here is a hypothesis. The golden set from module 6 is what turns 'this feels the same' into evidence that quality held.

The five things to carry forward

- ◆ **Everything is tokens.** Send fewer, generate fewer, or avoid the invocation.
- ◆ **Application inference profiles** attribute spend per team or tenant. Daily aggregate, not per request, and not retroactive.
- ◆ **Semantic caching** when humans reword. An exact-match hash sits near a zero hit rate.
- ◆ **Bursty traffic argues against every reservation.** Batch has no completion guarantee.
- ◆ **Three latency metrics.** TTFT for streaming, total latency for machines, throughput for peaks.

ON THE EXAM

Module 15 closes Domain 4 with monitoring and observability: what to measure, which CloudWatch metrics exist, and how to trace across the FM boundary.